Computational Complexity

Reading guide and Homework for **Week 03**

1 Reading Guide

In the second class we have learned:

- We saw a few different computational models (RA Turing Machines, RAM machines, simple programming language), and we argued that these models solve the same problems in polynomial time. There is no section in the book that covers this thoroughly.
- We introduced the class NP, and saw a few examples. Read §2.1.
- We learned about exhaustive search algorithms, and why they are not efficient. We learned about Gödel's problem and Nash's problem. This is not in the book, but at the end of your reading, you should read the related section §2.7, and the "Chapter Notes and History" section that follows it.
- We learned about reductions, and NP-completeness. Read §2.2.
- We proved the Cook-Levin Theorem, that 3SAT is NP-complete. Read §2.3.

There were several details that we covered very quickly, and which **must** be looked at carefully, in order to properly understand what is going on. This kind of familiarity will be necessary to understand what's being said in class.

2 Homework Exercises

Exercises are organized by the skill that is being trained. The test and exam questions will always be about testing one of the skills appearing in the homework.

Skill: *show that a given problem is in* P.

1. (Python dictionaries) An *associative array*, or *python dict*, is a partial function $F : \{0,1\}^* \to \{0,1\}^*$ with finite support. In other words, it is a finite set F of pairs $(x,y) \in \{0,1\}^* \times \{0,1\}^*$ such that for every x there exists at most one pair (x,y) in F. For any associate array F, let $m = m(F) = |F|$ denote $|F|$, the number of pairs in F, and let $\ell = \ell(F) = \max_{(x,y)\in F} |x| + |y|$ denote the maximum length of any pair in F. Let Dict_n denote the set of all finite partial maps with $m(F) \leq n$.

   Consider the following operations on associative arrays:

   (a) *Insertion*: given $F \in \mathsf{Dict}_n$, and a pair $(x,y) \in \{0,1\}^* \times \{0,1\}^*$, outputs $F' = F \cup \{(x,y)\} \in \mathsf{Dict}_{n+1}$.

   (b) *Deletion*: given $F \in \mathsf{Dict}_n$, and x such that $(x,y) \in F$ for some y, outputs $F' = F \setminus \{(x,y)\} \in \mathsf{Dict}_{n-1}$.

   (c) *Replacement*: given $F \in \mathsf{Dict}_n$, x such that $(x,y) \in F$ for some y, and y', outputs $F' = (F \setminus \{(x,y)\}) \cup \{(x,y')\} \in \mathsf{Dict}_n$.

   (d) *Query*: given $F \in \mathsf{Dict}_n$, $x \in \{0,1\}^*$, output $b \in \{0,1\}$ such that b indicates whether there exist some $y \in \{0,1\}^*$ such that $(x,y) \in F$, and in case $b = 1$, also outputs such a y.

Show that the following operations can be efficiently implemented:

(a) Show that they can all be implemented in polynomial time on a multitape Turing machine. Show, in fact, that queries and deletions can be made to run in time $O(m(F)\ell(F)+|x|)$, and inserts in time $O(m(F)\ell(f)+|x|+|y|)$. Here insertions and deletions are given F in the first tape and must output F' in the last tape.

(b) Now suppose that $\ell(F) \leq \log n$, i.e., we only wish to store small numbers. Then show that, under a clever encoding, a Random-Acess Machine (RAM) can implement the above operations in time $O(\log n)$, provided that insertions and deletions are allowed to destructively modify F to obtain F', instead of creating a new copy. [*Hint:* lookup *associative array*, and *self-balancing binary search tree*, on wikipedia]

2. Note that the state of memory of a random-access machine is itself an associative array. Convince yourself that it must then follow from the above that polynomial time on a RAM can be simulated in polynomial time on a multitape Turing machine.

3. Show that 2SAT, the problem of deciding whether a 2-CNF is satisfiable, is in P.

Skill: *show that a given problem is in* NP.

4. Show that deciding whether a given graph G has a k-clique, when given both G and k as input (so k is not fixed), is in NP.[Hint]

5. Do exercises 2.2 to 2.5 in the book (2COL, 3COL, CONNECTED, LINEQ, Linear Programming, PRIMES).

6. Show that if $A \subseteq \{0,1\}^*$ is in NP and $B \subseteq \{0,1\}^*$ is in NP, then $A \cap B$ and $A \cup B$ are in NP.

Skill: *show that a given problem is* NP*-complete.*

7. Consider the problem of deciding whether a given system of multivariate polynomial equations with integer coefficients has a Boolean solution. Meaning, we are given as input a system of polynomial equations with integer coefficients in n-variables. One such equation, for example, is $4x_1 - 8x_2^2 + 9x_1x_2 = 0$, and we are given several of them. We wish to decide whether there exists some setting of 0/1 to the variables $x_1, \ldots, x_n$, so that all given equations are satisfied simultaneously.

   Show that BPEqs is NP-complete. *Hint:* can you construct, given a 3CNF, a system of such equations, which has a Boolean solution if and only if the given 3CNF is satisfiable?

[Hint] A set of vertices witnesses that G has a k clique.